Design and Implementation of a Single-Cycle RV32I RISC-V Processor Using SystemVerilog

Muhammad Ahmed Raza

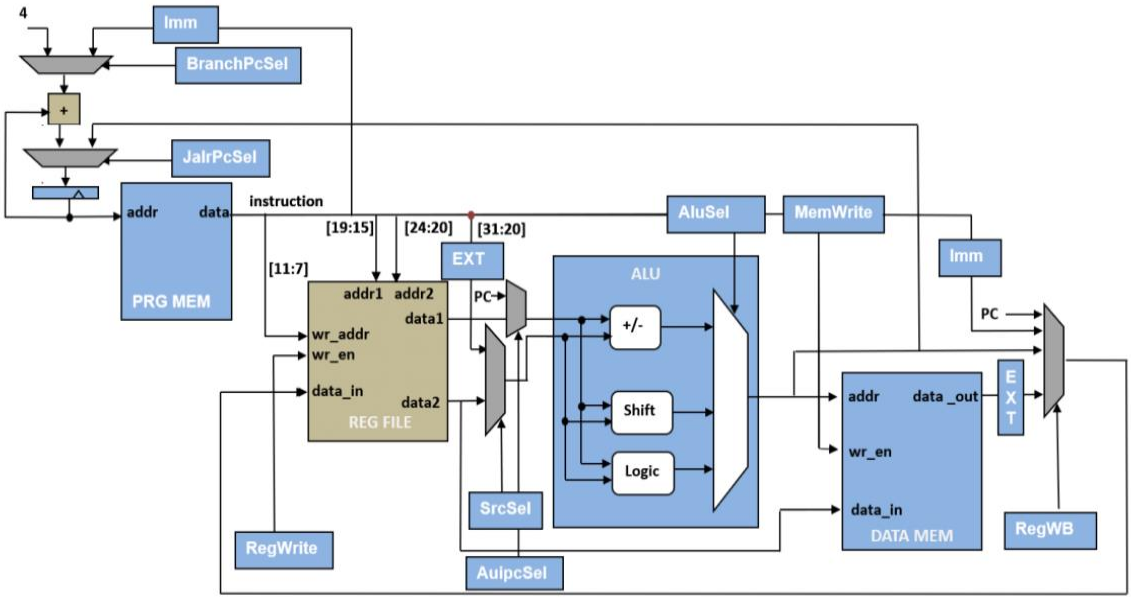
Department of Computer Science, Abasyn University, Islamabad, PAKISTAN

razaanjum9605@gmail.com

Abstract

The RISC-V processor has an open instruction set. Everyone can use it to make a processor. But the main thing is to understand step by step how this processor actually works. In my recent work, the most entertaining and interesting work I have done so far in my engineering career is making this processor with some basic building blocks. Initially, it was very tough to understand. Honestly, I saw some of my colleagues quit because of its complexity, but I believe that if you spend time on any problem, you can do it. It was scary and unbelievable when we heard that we were going to make a processor. But with the name of Allah Almighty and by working day and night, we finally did it. By going through this process, I faced a lot of difficulties with designing, and that is why it is more memorable for me. It was quite an impressive and enjoyable journey, and I want to document it; maybe it will help someone.

Design of RV32I Processor



Module Names:

Following are the names which I used to make this processor:

- Program Counter
- Instruction Memory
- Registers File
- ALU (Arithmetic Logic Unit)
- Data Memory
- Immediate Generator
- Control Unit
- RV32I Top
- RV32I Top Testbench

Program Counter

```
module Program_Counter (
input logic clk,
input logic reset,
input logic [31:0] pc_next,
output logic [31:0] pc
);
always_ff @(posedge clk)
begin
if (reset)
pc <= 32'd0;
else
pc <= pc_next;
end
endmodule
```

Instruction Memory

```
module InstrMem#(
parameter ADDR_BITS = 10,
parameter DATA_FILE = "imem.hex"
 )(
```

```
input logic [31:0] addr,
output logic [31:0] instruction
 );
localparam DEPTH = 2**ADDR_BITS;
logic [31:0] mem [0:DEPTH-1];
initial begin $readmemh(DATA_FILE, mem);
end
assign instruction = mem[addr[ADDR_BITS+1:2]];
endmodule
```

Registers File

```
module reg_file (
    input  logic        clk,
    input  logic        reset,

    input  logic [4:0]  rs1,
    input  logic [4:0]  rs2,

    output logic [31:0] rd1,
    output logic [31:0] rd2,

    input  logic [4:0]  rd,
    input  logic [31:0] wr_data,
```

```
    input  logic        reg_write
);

    logic [31:0] r0;
    logic [31:0] r1;
    logic [31:0] r2;
    logic [31:0] r3;
    logic [31:0] r4;
    logic [31:0] r5;
    logic [31:0] r6;
    logic [31:0] r7;
    logic [31:0] r8;
    logic [31:0] r9;
    logic [31:0] r10;
    logic [31:0] r11;
    logic [31:0] r12;
    logic [31:0] r13;
    logic [31:0] r14;
    logic [31:0] r15;
    logic [31:0] r16;
    logic [31:0] r17;
    logic [31:0] r18;
    logic [31:0] r19;
    logic [31:0] r20;
```

```
    logic [31:0] r21;
    logic [31:0] r22;
    logic [31:0] r23;
    logic [31:0] r24;
    logic [31:0] r25;
    logic [31:0] r26;
    logic [31:0] r27;
    logic [31:0] r28;
    logic [31:0] r29;
    logic [31:0] r30;
    logic [31:0] r31;

    assign r0 = 32'd0;


    logic wen0,  wen1,  wen2,  wen3,  wen4,  wen5,
wen6,  wen7;
    logic wen8,  wen9,  wen10, wen11, wen12, wen13,
wen14, wen15;
    logic wen16, wen17, wen18, wen19, wen20, wen21,
wen22, wen23;
    logic wen24, wen25, wen26, wen27, wen28, wen29,
wen30, wen31;

    assign wen0  = 1'b0;
    assign wen1  = reg_write & (rd == 5'd1);
```

```
assign wen2  = reg_write & (rd == 5'd2);
assign wen3  = reg_write & (rd == 5'd3);
assign wen4  = reg_write & (rd == 5'd4);
assign wen5  = reg_write & (rd == 5'd5);
assign wen6  = reg_write & (rd == 5'd6);
assign wen7  = reg_write & (rd == 5'd7);
assign wen8  = reg_write & (rd == 5'd8);
assign wen9  = reg_write & (rd == 5'd9);
assign wen10 = reg_write & (rd == 5'd10);
assign wen11 = reg_write & (rd == 5'd11);
assign wen12 = reg_write & (rd == 5'd12);
assign wen13 = reg_write & (rd == 5'd13);
assign wen14 = reg_write & (rd == 5'd14);
assign wen15 = reg_write & (rd == 5'd15);
assign wen16 = reg_write & (rd == 5'd16);
assign wen17 = reg_write & (rd == 5'd17);
assign wen18 = reg_write & (rd == 5'd18);
assign wen19 = reg_write & (rd == 5'd19);
assign wen20 = reg_write & (rd == 5'd20);
assign wen21 = reg_write & (rd == 5'd21);
assign wen22 = reg_write & (rd == 5'd22);
assign wen23 = reg_write & (rd == 5'd23);
assign wen24 = reg_write & (rd == 5'd24);
assign wen25 = reg_write & (rd == 5'd25);
```

```
assign wen26 = reg_write & (rd == 5'd26);
assign wen27 = reg_write & (rd == 5'd27);
assign wen28 = reg_write & (rd == 5'd28);
assign wen29 = reg_write & (rd == 5'd29);
assign wen30 = reg_write & (rd == 5'd30);
assign wen31 = reg_write & (rd == 5'd31);



always_ff @(posedge clk)
begin if (reset) r1  <= 32'd0;
 else if (wen1)  r1  <= wr_data;
 end

always_ff @(posedge clk)
begin if (reset) r2  <= 32'd0;
else if (wen2)  r2  <= wr_data;
end

always_ff @(posedge clk)
begin if (reset) r3  <= 32'd0;
else if (wen3)  r3  <= wr_data;
end

always_ff @(posedge clk)
```

```
begin if (reset) r4  <= 32'd0;
else if (wen4)  r4  <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r5  <= 32'd0;
else if (wen5)  r5  <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r6  <= 32'd0;
else if (wen6)  r6  <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r7  <= 32'd0;
else if (wen7)  r7  <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r8  <= 32'd0;
else if (wen8)  r8  <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r9  <= 32'd0;
else if (wen9)  r9  <= wr_data;
end
always_ff @(posedge clk)
```

```
begin if (reset) r10 <= 32'd0;
else if (wen10) r10 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r11 <= 32'd0;
else if (wen11) r11 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r12 <= 32'd0;
else if (wen12) r12 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r13 <= 32'd0;
else if (wen13) r13 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r14 <= 32'd0;
else if (wen14) r14 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r15 <= 32'd0;
else if (wen15) r15 <= wr_data;
end
always_ff @(posedge clk)
```

```
begin if (reset) r16 <= 32'd0;
else if (wen16) r16 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r17 <= 32'd0;
else if (wen17) r17 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r18 <= 32'd0;
else if (wen18) r18 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r19 <= 32'd0;
else if (wen19) r19 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r20 <= 32'd0;
else if (wen20) r20 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r21 <= 32'd0;
else if (wen21) r21 <= wr_data;
end
always_ff @(posedge clk)
```

```
begin if (reset) r22 <= 32'd0;
else if (wen22) r22 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r23 <= 32'd0;
else if (wen23) r23 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r24 <= 32'd0;
else if (wen24) r24 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r25 <= 32'd0;
else if (wen25) r25 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r26 <= 32'd0;
else if (wen26) r26 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r27 <= 32'd0;
else if (wen27) r27 <= wr_data;
end
always_ff @(posedge clk)
```

```
begin if (reset) r28 <= 32'd0;
else if (wen28) r28 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r29 <= 32'd0;
else if (wen29) r29 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r30 <= 32'd0;
else if (wen30) r30 <= wr_data;
end
always_ff @(posedge clk)
begin if (reset) r31 <= 32'd0;
else if (wen31) r31 <= wr_data;
end

always_comb
begin
    case (rs1)
        5'd0  : rd1 = r0;
        5'd1  : rd1 = r1;
        5'd2  : rd1 = r2;
        5'd3  : rd1 = r3;
        5'd4  : rd1 = r4;
```

```
5'd5  : rd1 = r5;
5'd6  : rd1 = r6;
5'd7  : rd1 = r7;
5'd8  : rd1 = r8;
5'd9  : rd1 = r9;
5'd10 : rd1 = r10;
5'd11 : rd1 = r11;
5'd12 : rd1 = r12;
5'd13 : rd1 = r13;
5'd14 : rd1 = r14;
5'd15 : rd1 = r15;
5'd16 : rd1 = r16;
5'd17 : rd1 = r17;
5'd18 : rd1 = r18;
5'd19 : rd1 = r19;
5'd20 : rd1 = r20;
5'd21 : rd1 = r21;
5'd22 : rd1 = r22;
5'd23 : rd1 = r23;
5'd24 : rd1 = r24;
5'd25 : rd1 = r25;
5'd26 : rd1 = r26;
5'd27 : rd1 = r27;
5'd28 : rd1 = r28;
```

```
        5'd29 : rd1 = r29;
        5'd30 : rd1 = r30;
        5'd31 : rd1 = r31;
        default : rd1 = 32'd0;
    endcase
end

always_comb
begin
    case (rs2)
        5'd0  : rd2 = r0;
        5'd1  : rd2 = r1;
        5'd2  : rd2 = r2;
        5'd3  : rd2 = r3;
        5'd4  : rd2 = r4;
        5'd5  : rd2 = r5;
        5'd6  : rd2 = r6;
        5'd7  : rd2 = r7;
        5'd8  : rd2 = r8;
        5'd9  : rd2 = r9;
        5'd10 : rd2 = r10;
        5'd11 : rd2 = r11;
        5'd12 : rd2 = r12;
        5'd13 : rd2 = r13;
```

```
            5'd14 : rd2 = r14;
            5'd15 : rd2 = r15;
            5'd16 : rd2 = r16;
            5'd17 : rd2 = r17;
            5'd18 : rd2 = r18;
            5'd19 : rd2 = r19;
            5'd20 : rd2 = r20;
            5'd21 : rd2 = r21;
            5'd22 : rd2 = r22;
            5'd23 : rd2 = r23;
            5'd24 : rd2 = r24;
            5'd25 : rd2 = r25;
            5'd26 : rd2 = r26;
            5'd27 : rd2 = r27;
            5'd28 : rd2 = r28;
            5'd29 : rd2 = r29;
            5'd30 : rd2 = r30;
            5'd31 : rd2 = r31;
            default : rd2 = 32'd0;
        endcase
    end

endmodule
```

ALU (Arithmetic Logic Unit)

```
module ALU (
input logic [31:0] A,
input logic [31:0] B,
input logic [3:0] ALUSel,
output logic [31:0] ALU_Result,
output logic Zero
);

logic [4:0] shamt;

always_comb
begin
shamt = B[4:0];
ALU_Result = 32'd0;

case(ALUSel)
4'b0000: ALU_Result = A + B;
4'b0001: ALU_Result = A - B;
4'b0010: ALU_Result = A & B;
4'b0011: ALU_Result = A | B;
4'b0100: ALU_Result = A ^ B;
```

```
4'b0101: ALU_Result = A << shamt;
4'b0110: ALU_Result = A >> shamt;
4'b0111: ALU_Result = $signed(A) >>> shamt;
4'b1000: ALU_Result = {31'b0, ($signed(A) <
$signed(B))};
4'b1001: ALU_Result = {31'b0, (A < B)};
default: ALU_Result = 32'd0;
endcase
end

assign Zero = (ALU_Result == 0);

endmodule
```

Data Memory

```
module DataMem #(
parameter ADDR_BITS = 10,
parameter DATA_FILE = "dmem.txt"
)(
input logic clk,
input logic mem_write,
input logic [31:0] addr,
input logic [31:0] write_data,
```

```
output logic [31:0] read_data
);
localparam DEPTH = 2**ADDR_BITS;
logic [31:0] mem [0:DEPTH-1];
initial begin
$readmemh(DATA_FILE, mem);
end
always_ff @(posedge clk)
begin
if(mem_write)
mem[addr[ADDR_BITS+1:2]] <= write_data;
end
assign read_data = mem[addr[ADDR_BITS+1:2]];

endmodule
```

Immediate Generator

```
module ImmGen (
    input  logic [31:0] instruction,
    output logic [31:0] imm
);

logic [6:0] op;
```

```
always_comb begin
    op  = instruction[6:0];
    imm = 32'd0;

    case(op)

    // I-Type
    7'b0010011,
    7'b0000011,
    7'b1100111: begin
        imm = {{20{instruction[31]}},
instruction[31:20]};
    end

    // S-Type
    7'b0100011: begin
        imm = {{20{instruction[31]}},
instruction[31:25], instruction[11:7]};
    end

    // B-Type
    7'b1100011: begin
        imm = {{19{instruction[31]}},
               instruction[31],
```

```
            instruction[7],
            instruction[30:25],
            instruction[11:8],
            1'b0};
end

// U-Type
7'b0110111,
7'b0010111: begin
    imm = {instruction[31:12], 12'b0};
end

// J-Type
7'b1101111: begin
    imm = {{11{instruction[31]}},
            instruction[31],
            instruction[19:12],
            instruction[20],
            instruction[30:21],
            1'b0};
end

default: begin
    imm = 32'd0;
```

```
    end

    endcase
end

endmodule
```

Control Unit

```
module ControlUnit (
input logic [6:0] opcode,
input logic [2:0] funct3,
input logic [6:0] funct7,
output logic RegWrite,
output logic MemWrite,
output logic Branch,
output logic Jump,
output logic ALUSrc,
output logic [1:0] ResultSrc,
output logic [3:0] ALUSel
);

logic f7_5;
```

```
assign f7_5 = funct7[5];

always_comb
begin

RegWrite = 0;
MemWrite = 0;
Branch = 0;
Jump = 0;
ALUSrc = 0;
ResultSrc = 2'b00;
ALUSel = 4'b0000;

case(opcode)

// R-Type
7'b0110011: begin
RegWrite = 1;
ALUSrc = 0;
case(funct3)
3'b000: ALUSel = (f7_5) ? 4'b0001 : 4'b0000;
3'b111: ALUSel = 4'b0010;
3'b110: ALUSel = 4'b0011;
3'b100: ALUSel = 4'b0100;
```

```
3'b001: ALUSel = 4'b0101;
3'b101: ALUSel = (f7_5) ? 4'b0111 : 4'b0110;
3'b010: ALUSel = 4'b1000;
3'b011: ALUSel = 4'b1001;
default: ALUSel = 4'b0000;
endcase
end

// I-Type
7'b0010011: begin
RegWrite = 1;
ALUSrc = 1;
case(funct3)
3'b000: ALUSel = 4'b0000;
3'b111: ALUSel = 4'b0010;
3'b110: ALUSel = 4'b0011;
3'b100: ALUSel = 4'b0100;
3'b001: ALUSel = 4'b0101;
3'b101: ALUSel = (f7_5) ? 4'b0111 : 4'b0110;
3'b010: ALUSel = 4'b1000;
3'b011: ALUSel = 4'b1001;
default: ALUSel = 4'b0000;
endcase
end
```

```
// Load
7'b0000011: begin
RegWrite = 1;
ALUSrc = 1;
ResultSrc = 2'b01;
ALUSel = 4'b0000;
end

// Store
7'b0100011: begin
MemWrite = 1;
ALUSrc = 1;
ALUSel = 4'b0000;
end

// Branch
7'b1100011: begin
Branch = 1;
ALUSel = 4'b0001;
end

// JAL
7'b1101111: begin
```

```
Jump = 1;
RegWrite = 1;
ResultSrc = 2'b10;
end

// JALR
7'b1100111: begin
RegWrite  = 1;
ALUSrc    = 1;
Jump      = 1;
ResultSrc = 2'b10;
ALUSel    = 4'b0000;
end

// LUI
7'b0110111: begin
RegWrite = 1;
ResultSrc = 2'b11;
end

default: begin
RegWrite = 0;
MemWrite = 0;
Branch = 0;
```

```
Jump = 0;
ALUSrc = 0;
ResultSrc = 2'b00;
ALUSel = 4'b0000;
end

endcase
end

endmodule
```

RV321 Top Module

```
module RV32I_Top (
    input logic clk,
    input logic reset
);

    logic [31:0] pc;
    logic [31:0] pc_next;
    logic [31:0] instruction;
    logic [31:0] imm;
    logic [4:0]  rs1, rs2, rd;
    logic [6:0]  opcode;
```

```
logic [2:0]  funct3;
logic [6:0]  funct7;
logic [31:0] rd1, rd2;
logic [31:0] alu_in2;
logic [31:0] alu_result;
logic [31:0] mem_data;
logic [31:0] writeback_data;
logic        RegWrite;
logic        MemWrite;
logic        Branch;
logic        Jump;
logic        ALUSrc;
logic [1:0]  ResultSrc;
logic [3:0]  ALUSel;
logic        Zero;
logic        branch_taken;

assign opcode = instruction[6:0];
assign rd     = instruction[11:7];
assign funct3 = instruction[14:12];
assign rs1    = instruction[19:15];
assign rs2    = instruction[24:20];
assign funct7 = instruction[31:25];
```

```
Program_Counter PC (
    .clk    (clk),
    .reset  (reset),
    .pc_next(pc_next),
    .pc     (pc)
);

InstrMem IMEM (
    .addr       (pc),
    .instruction(instruction)
);

reg_file RF (
    .clk      (clk),
    .reset    (reset),
    .rs1      (rs1),
    .rs2      (rs2),
    .rd1      (rd1),
    .rd2      (rd2),
    .rd       (rd),
    .wr_data  (writeback_data),
    .reg_write(RegWrite)
);
```

```
ImmGen IG (
    .instruction(instruction),
    .imm         (imm)
);

ControlUnit CU (
    .opcode   (opcode),
    .funct3   (funct3),
    .funct7   (funct7),
    .RegWrite (RegWrite),
    .MemWrite (MemWrite),
    .Branch   (Branch),
    .Jump     (Jump),
    .ALUSrc   (ALUSrc),
    .ResultSrc(ResultSrc),
    .ALUSel   (ALUSel)
);

assign alu_in2 = ALUSrc ? imm : rd2;

ALU alu (
    .A         (rd1),
    .B         (alu_in2),
    .ALUSel    (ALUSel),
```

```
        .ALU_Result(alu_result),
        .Zero      (Zero)
    );

    DataMem DMEM (
        .clk       (clk),
        .mem_write (MemWrite),
        .addr      (alu_result),
        .write_data(rd2),
        .read_data (mem_data)
    );

    always_comb
    begin
        case (ResultSrc)
            2'b00:   writeback_data = alu_result;
            2'b01:   writeback_data = mem_data;
            2'b10:   writeback_data = pc + 32'd4;
            2'b11:   writeback_data = imm;
            default: writeback_data = 32'd0;
        endcase
    end

always_comb begin
```

```
    branch_taken = 1'b0;

    case (funct3)

        3'b000: branch_taken = (rd1 == rd2);

        3'b001: branch_taken = (rd1 != rd2);
        3'b100: branch_taken = ($signed(rd1) <
$signed(rd2));

        3'b101: branch_taken = ($signed(rd1) >=
$signed(rd2));

        3'b110: branch_taken = (rd1 < rd2);

        3'b111: branch_taken = (rd1 >= rd2);

    endcase
end

    always_comb
    begin
    pc_next = pc + 4;

    if (Jump && opcode == 7'b1100111)
```

```
        pc_next = (rd1 + imm) & 32'hFFFFFFFE;

    else if (Jump)
        pc_next = pc + imm;

    else if (Branch && branch_taken)
        pc_next = pc + imm;
end

endmodule
```

RV32I Top Testbench Module

```
`timescale 1ns/1ps

module RV32I_Top_tb;

logic clk;
logic reset;

RV32I_Top uut (
    .clk(clk),
    .reset(reset)
);
```

```
always #5 clk = ~clk;

initial begin
$dumpfile("waveform.vcd");
$dumpvars(0,RV32I_Top_tb );

clk = 0;
reset = 1;

#10 reset = 0;

#500 $finish;

end

initial begin
$monitor(
"Time=%0t PC=%h Instruction=%h ALU_Result=%h
WriteBack=%h",
$time,
uut.pc,
uut.instruction,
uut.alu_result,
uut.writeback_data
```

```
);
end

endmodule
```

Machine Code and its Assembly Code

Following is a machine code in hexadecimal:

00500093

00500113

00208463

The translation of this above code in assembly language is given below:

addi x1, x0, 5 // It means load 5 in x1.

addi x2, x0, 5 // It means load 5 in x2

beq x1, x2, 8 // It means if the values of x1 and x2 are equal then forward by 8 bytes

Simulation Commands

```
iverilog -g2012 -o sim.out rv32i_top_tb.sv rv32i_top.sv
program_counter.sv instr_mem.sv reg_file.sv imm_gen.sv
control_unit.sv alu.sv data_mem.sv
```

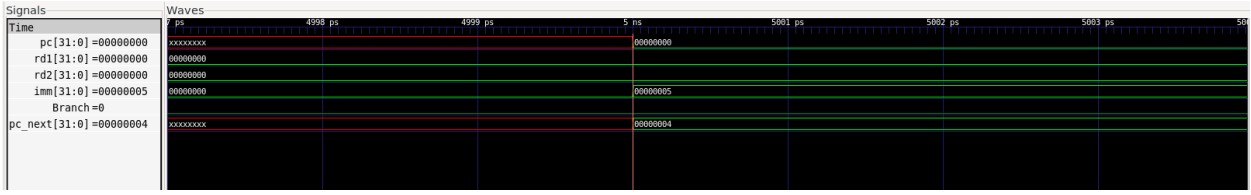
```
vvp sim.out
```

```
gtkwave waveform.vcd
```

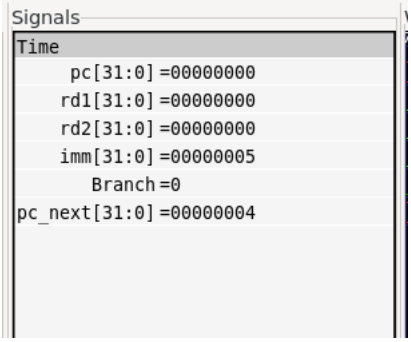
Wave Results

As we all know, this processor is based on a 32-bit instruction set, which means it increments 4 every time.

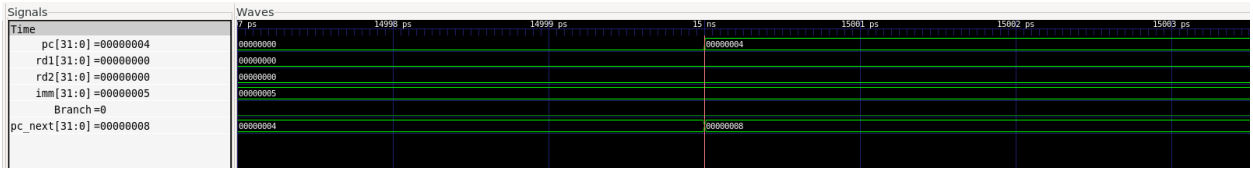
At the first-time increment,



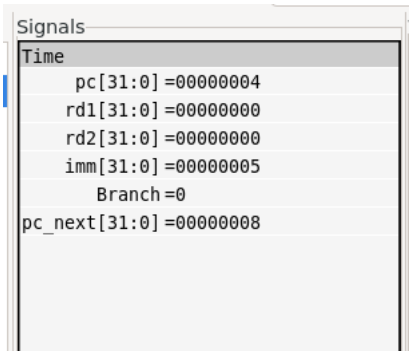
Here you can see PC = 0, which means it is at the initial stage, and it loads 5. Imm = 5. Because 5 is constant, we can also call it immediate. It means our first instruction was executed successfully.



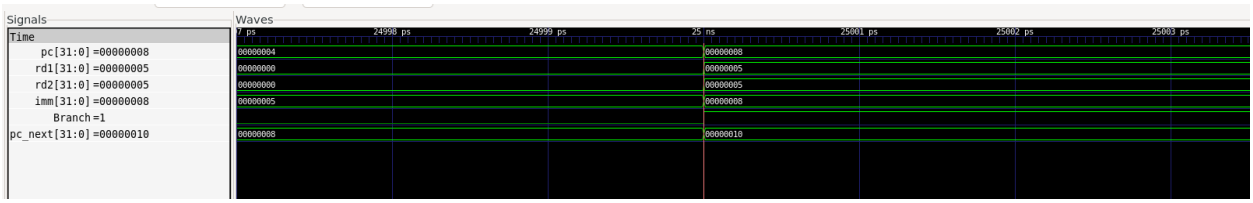
At Second time Increment,



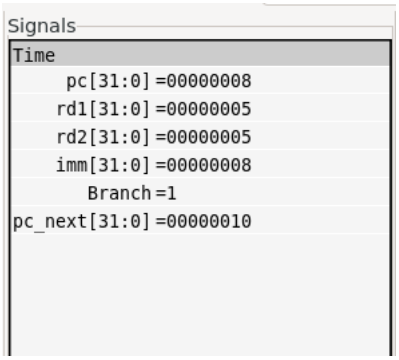
Now you can see PC = 4, which means it was incremented by 4, and in the next round it loads 5 again. Why? Because I gave it the machine code that I wrote earlier, which says to load 5 the first time, then load 5 again, and after that compare both; if equal, move forward by 8.



At Third time Increment,



In this, if you see rd1 and rd2, both have number 5. It means both are equal, so it will jump ahead by 8 bytes. In the immediate, it is 8 and branch is also 1, which means the condition is true.



Conclusion:

A Single-Cycle RV32I RISC-V processor was successfully implemented using SystemVerilog. The processor executed instructions correctly and demonstrated the functionality of key components such as the Register File, ALU, Control Unit, and Memory. The project provided valuable experience in processor design and verification.